

React `useEffect` – Notes

1. PUT vs PATCH (Quick Context)

PUT

- Updates the **entire resource**
- All fields must be sent
- Missing fields may get overwritten

PATCH

- Updates **specific fields only**
 - Common use case: forgot password
 - Safer for partial updates
-

2. Why `useEffect` Exists

React components:

- Render UI first
- Perform side effects **after render**

`useEffect` is used for:

- API calls
 - Data fetching
 - Subscriptions
 - Side effects
-

3. Basic Syntax

```
useEffect(() => {  
  // side effect  
}, []);
```

Dependency Array

- `[]` → runs **only once**
 - Used for initial data fetching
 - Prevents repeated API calls
-

4. Fetching Data with `useEffect`

```
useEffect(() => {
  const fetchURL = async () => {
    try {
      const response = await fetch("https://jsonplaceholder.typicode.com/posts");
      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message || "Something went wrong");
      }

      setUrl(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  fetchURL();
}, []);
```

Explanation

- `try` → successful request
- `catch` → error handling
- `finally` → stop loading

5. UI Representation of Background Work

State Variables

```
const [url, setUrl] = useState([]);
const [error, setError] = useState("");
const [isLoading, setLoading] = useState(true);
```

Conditional Rendering

```
{isLoading && <p>Loading...</p>}
{error && <p>{error}</p>}
```

- Shows loading while fetching
- Shows error if API fails

6. Edge Case Handling

Empty API Response

```
{url.length === 0 ? (
  <p>No URL data found.</p>
) : (
  // render list
)}
```

Prevents blank or broken UI.

7. Rendering Lists Using `map`

```
{Url.map(url => (  
  <li key={url.id}>  
    <h2>{url.title}</h2>  
    <p>{url.body}</p>  
  </li>  
))}
```

8. Importance of `key`

Without `key` :

- React compares items by **position**
- Entire list may re-render

With `key` :

- React compares by **identity**
- Only changed item updates

Example

```
key = 1,2,3  
  
{a:1,b:2,c:3}  
  
{d,a:1,m,c:3}
```

Without keys → full re-render With keys → only changed item updates

9. Complete Component

```

import React, { useState, useEffect } from "react";

const Url = () => {
  const [Url, setUrl] = useState([]);
  const [error, setError] = useState("");
  const [isLoading, setLoading] = useState(true);

  useEffect(() => {
    const fetchURL = async () => {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/posts"
        );
        const data = await response.json();

        if (!response.ok) {
          throw new Error(data.message || "Something went wrong");
        }

        setUrl(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchURL();
  }, []);

  return (
    <>
      <h1>URL DATA</h1>

      {isLoading && <p>Loading...</p>}
      {error && <p>{error}</p>}

      {!isLoading && !error && (
        <div>
          {Url.length === 0 ? (
            <p>No URL data found.</p>
          ) : (
            <ul>
              {Url.map((url) => (
                <li key={url.id}>
                  <h2>{url.title}</h2>
                  <p>{url.body}</p>
                </li>
              ))}
            </ul>
          )}
        </div>
      )}
    </>
  );
};

export default Url;

```

10. Key Takeaways

- `useEffect` handles side effects

- Empty dependency array → run once
 - Always manage:
 - Loading state
 - Error state
 - Empty data
 - Use `key` when rendering lists
 - React updates only changed elements when keys exist
-

```
const url = [  
  {  
    url: "https://www.google.com",  
    status: "200 OK",  
    responseTime: "150ms",  
  },  
  {  
    url: "https://www.facebook.com",  
    status: "200 OK",  
    responseTime: "100ms",  
  },  
  {  
    url: "https://www.instagram.com",  
    status: "200 OK",  
    responseTime: "120ms",  
  },  
  {  
    url: "https://www.twitter.com",  
    status: "200 OK",  
    responseTime: "80ms",  
  },  
  {  
    url: "https://www.linkedin.com",  
    status: "200 OK",  
    responseTime: "90ms",  
  },  
  {  
    url: "https://www.github.com",  
    status: "200 OK",  
    responseTime: "110ms",  
  },  
];
```